

VIII: Introduction aux Réseaux de Neurones

Guillaume Guex guillaume.guex@unil.ch

Des modèles linéaires aux réseaux de neurones

Régression linéaire

Le but de la **régression linéaire** (sans constante pour le moment) est d'estimer une variable numérique Y avec les variables numériques $X_1, \dots, X_p$. On a :

$$\hat{y}_i := w_1 x_{i1} + \dots + w_p x_{ip}$$

où $\hat{y}_i$ est une **prédiction** et x_{ij} est la valeur de l'individu i sur la variable j .

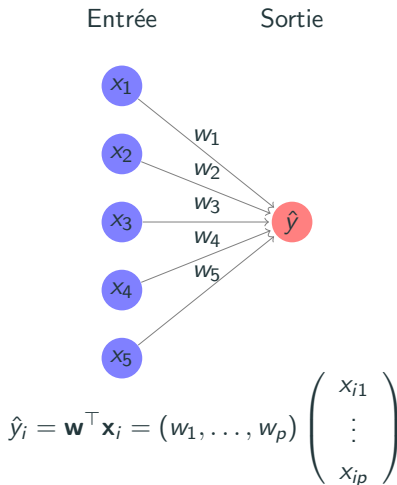
Les **coefficients** $w_1, \dots, w_p$ s'obtiennent en minimisant l'**erreur carrée moyenne** :

$$MSE(w_0, w_1, \dots, w_p) = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

où y_i est la vraie valeur de Y pour l'individu i .

Régression linéaire : réseau de neurones

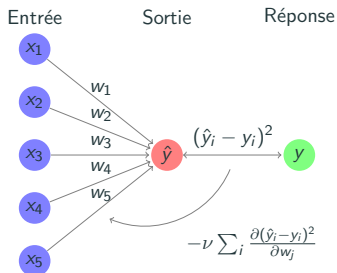
En réalité, le modèle de régression multiple peut être vu comme un **réseau de neurones**. Dans ce cadre, les w_j s'appellent les **poids**.



Régression linéaire : réseau de neurones

On peut trouver la solution par **descente du gradient**. Pour chaque itération :

- On passe **séquentiellement** $(\mathbf{x}_1, \dots, \mathbf{x}_n)$ dans le réseau.
- On calcule les **pertes** $L_i(\mathbf{w}) = (\hat{y}_i - y_i)^2$.
- On calcule les **gradients** $\frac{\partial L_i(\mathbf{w})}{\partial w_j}$.
- On **ajuste les poids** avec $w_j = w_j - \nu \sum_i \frac{\partial L_i(\mathbf{w})}{\partial w_j}$.

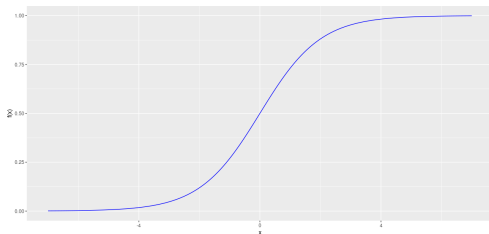


Régression logistique

Avec la **régression logistique**, on cherche à estimer une **variable binaire** Y avec $X_1, \dots, X_n$. Une **prédiction** $\hat{y}_i$ donnée par

$$\hat{y}_i = \sigma(w_1 x_{i1} + \dots + w_p x_{ip})$$

où $\sigma(x) = \frac{1}{1+\exp(-x)}$ est la **fonction sigmoïde**.

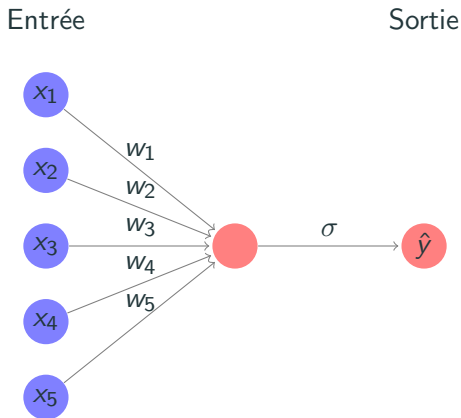


Ici, on minimise l'**entropie croisée** :

$$R(w_0, \dots, w_p) = - \sum_{i=1}^n (y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i))$$

Régression logistique : réseau de neurones

La régression logistique peut aussi être vu comme un **réseau de neurones** :

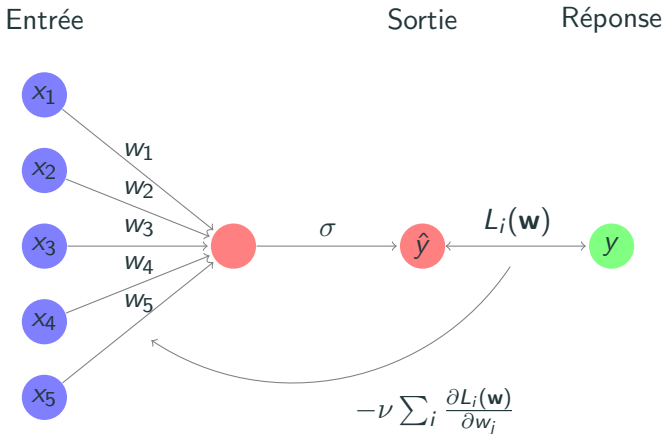


$$\hat{y}_i = \sigma(\mathbf{w}^\top \mathbf{x}_i)$$

Régression logistique : réseau de neurones

La **descente du gradient** peut aussi s'appliquer, avec

$$L_i(\mathbf{w}) = y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)$$



Multiple régression linéaires

Si l'on doit estimer q variables Y_k d'un objet i à partir de p entrées X_j , on peut effectuer q régressions linéaires :

$$\hat{y}_{i1} = w_{11}x_{i1} + w_{12}x_{i2} + \dots + w_{1p}x_{ip}$$

$$\hat{y}_{i2} = w_{21}x_{i1} + w_{22}x_{i2} + \dots + w_{2p}x_{ip}$$

$$\vdots$$

$$\hat{y}_{iq} = w_{q1}x_{i1} + w_{q2}x_{i2} + \dots + w_{qp}x_{ip}$$

Ce qui s'écrit, matriciellement :

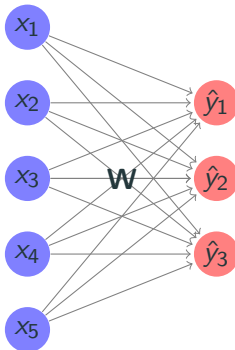
$$\hat{\mathbf{y}}_i = \mathbf{W}\mathbf{x}_i$$

$$\text{avec } \hat{\mathbf{y}}_i = \begin{pmatrix} \hat{y}_{i1} \\ \vdots \\ \hat{y}_{iq} \end{pmatrix}, \mathbf{x}_i = \begin{pmatrix} x_{i1} \\ \vdots \\ x_{ip} \end{pmatrix} \text{ et } \mathbf{W} = \begin{pmatrix} w_{11} & \cdots & w_{1p} \\ \vdots & \ddots & \vdots \\ w_{q1} & \cdots & w_{qp} \end{pmatrix}.$$

Multiple régression linéaires : réseau de neurones

Cela donne le **réseau de neurones** suivant :

Entrée Sortie



$$\hat{y}_i = \mathbf{W}\mathbf{x}_i$$

Que l'on peut encore optimiser par **descente du gradient**.

Régression logistique multinomiale

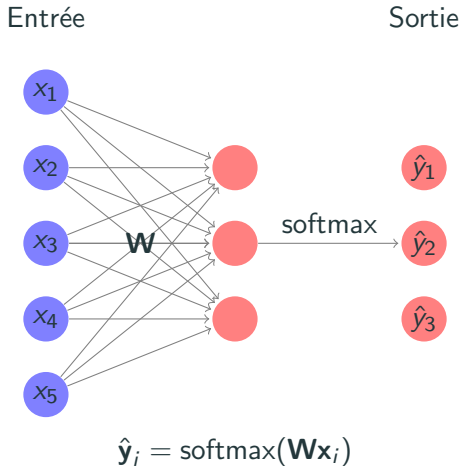
La **régression logistique multinomiale**, avec q classes, va trouver les **appartenances** $\hat{y}_{ik}$ de l'objet i dans la classe k . On les obtient avec q régressions et la fonction **softmax** :

$$\begin{pmatrix} \hat{y}_{i1} \\ \vdots \\ \hat{y}_{iq} \end{pmatrix} = \text{softmax} \begin{pmatrix} w_{11}x_{i1} + w_{12}x_{i2} + \dots + w_{1p}x_{ip} \\ \vdots \\ w_{q1}x_{i1} + w_{q2}x_{i2} + \dots + w_{qp}x_{ip} \end{pmatrix}$$
$$\hat{\mathbf{y}}_i = \text{softmax}(\mathbf{W}\mathbf{x}_i)$$

où la fonction $\text{softmax}(\mathbf{v})$ d'un vecteur $\mathbf{v}$ donne :

$$\text{softmax} \begin{pmatrix} v_1 \\ \vdots \\ v_k \end{pmatrix} = \begin{pmatrix} \frac{\exp(v_1)}{\exp(v_1) + \dots + \exp(v_k)} \\ \vdots \\ \frac{\exp(v_k)}{\exp(v_1) + \dots + \exp(v_k)} \end{pmatrix}$$

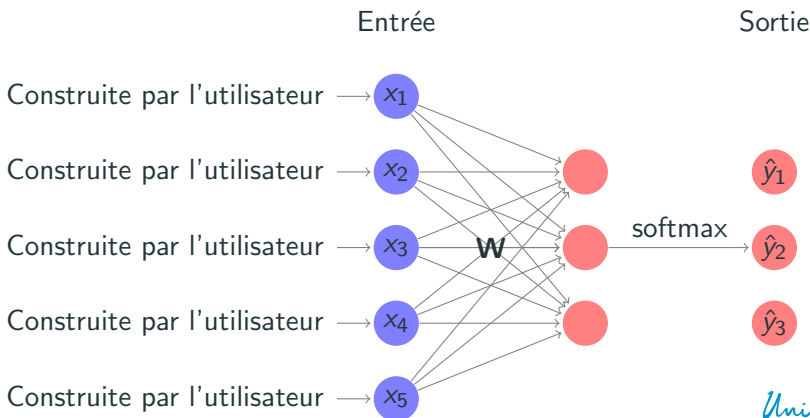
Régression logistique multinomiale : réseau de neurones



Encore une fois, la **descente du gradient** peut optimiser le système avec $L_i(\mathbf{W}) = -\sum_{k=1}^m y_{ik} \log(\hat{y}_{ik})$.

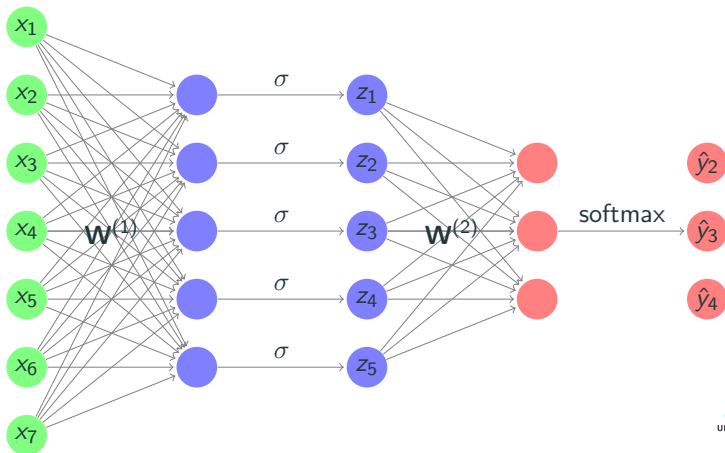
Les couches intermédiaires

Dans les modèles classiques, les caractéristiques d'entrées sont **construites par l'utilisateur**.



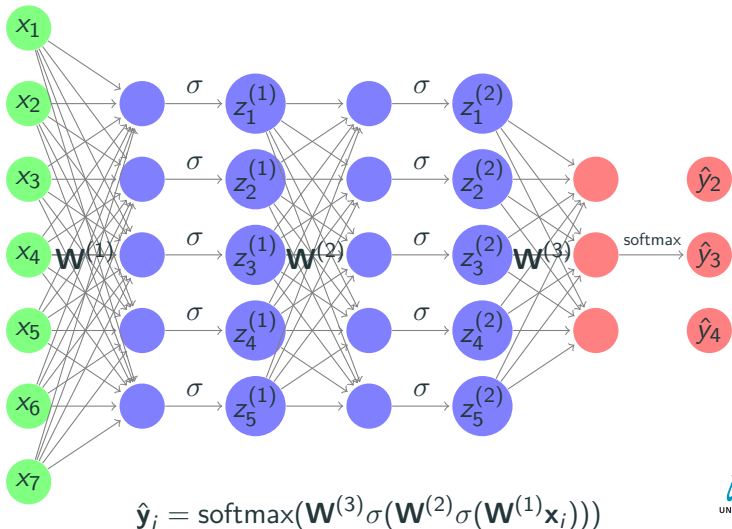
Les couches intermédiaires

On peut aussi commencer par faire des **régressions logistiques** qui sont **également entraînées** afin de trouver les **caractéristiques pertinentes de manière automatique**.



Les couches intermédiaires

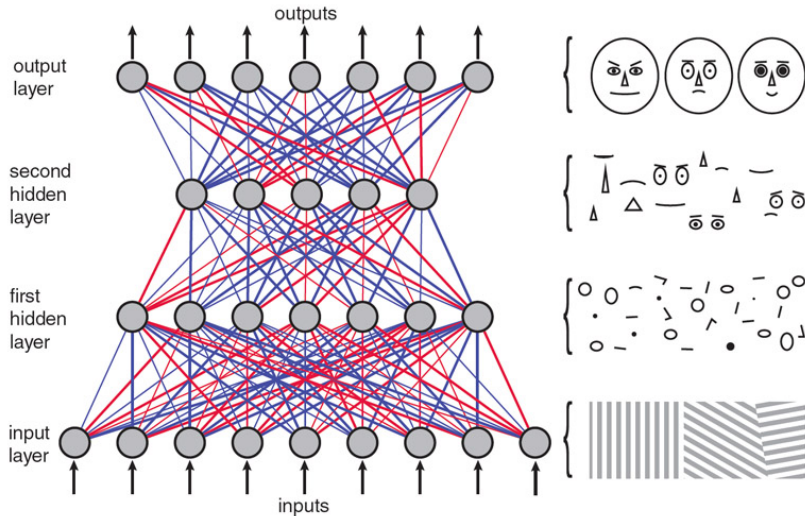
On peut même ajouter **plusieurs couches d'abstraction**.



Les **réseaux de neurones** (à propagation avant) se basent sur cette idée :

- On construit **plusieurs couches de neurones successives**.
- La couche d'entrée prend des **données brutes**, la couche de sortie donne nos **prédictions**.
- Toutes les couches sont **entraînées simultanément**.
- Une fois entraîné, plus on s'approche de la couche de sortie, plus le **degré d'abstraction** de la représentation est élevé.

Réseaux de neurones : principe général



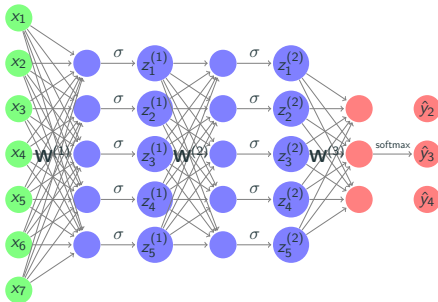
Source : Brian Hayes <https://www.americanscientist.org/article/delving-into-deep-learning>

Le perceptron multicouche

Le perceptron multicouche

Le **perceptron** est le premier réseau de neurones de l'histoire, inventé en 1943 par MxCulloch and Pitts et implémenté en 1958 par Rosenblatt. Il s'agit d'un réseau d'une seule couche.

Le **perceptron multicouche** est un réseau de neurones **multicouche**, **entièrement connecté** et à **propagation avant**.



L'utilisation des **fonctions sigmoïdes** σ fait que l'entrée de la couche suivante s'apparente à la **probabilité de l'objet de posséder une certaine caractéristique**.

Elles permettent aussi de **rompre la linéarité**. En effet, sans elles :

$$\begin{aligned}\hat{\mathbf{y}}_i &= \text{softmax}(\mathbf{W}^{(3)}\mathbf{W}^{(2)}\mathbf{W}^{(1)}\mathbf{x}_i) \\ &= \text{softmax}(\widetilde{\mathbf{W}}\mathbf{x}_i)\end{aligned}$$

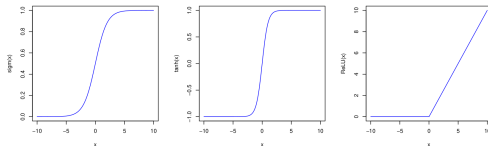
On les appelle des **fonctions d'activation**.

Les fonctions d'activations

En réalité, il y a **plusieurs possibilités** pour les fonctions d'activation.

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad \tanh(x) = \frac{2}{1 + e^{-2x}} - 1$$

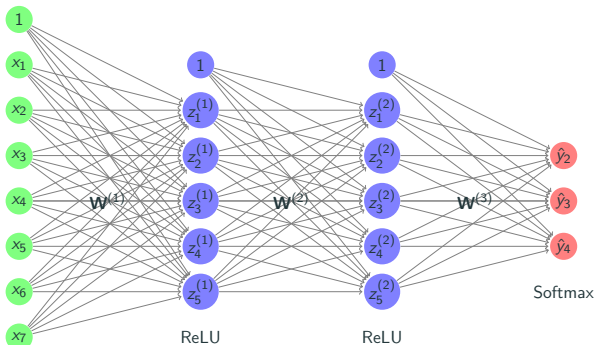
$$\text{ReLU}(x) = \begin{cases} 0 & x < 0 \\ x & x \geq 0 \end{cases}$$



La plus utilisée est le **ReLU**, où des dérivés plus modernes.

Le perceptron multicouche actuel

A chaque couche l , on peut aussi ajouter des **biais** $w_0^{(l)}$. Une représentation classique d'un modèle de **perceptron multicouche** est la suivante :



La **fonction activation finale** se définit en fonction de la tâche (Régression = aucune, Classification = Softmax).

L'entraînement

L'entraînement

On donne aux réseaux de neurones une **fonction de perte** qui s'écrit généralement comme :

$$L(\mathbf{W}) = \sum_i L_i(\mathbf{W})$$

où $\mathbf{W}$ dénote tous les **poids du réseaux**.

L'**entraînement** consiste à la minimiser grâce à un **optimisateur**, tel que la **descente du gradient** :

$$w(t+1) = w(t) - \nu \sum_i \frac{\partial L_i(\mathbf{W}(t))}{\partial w(t)}$$

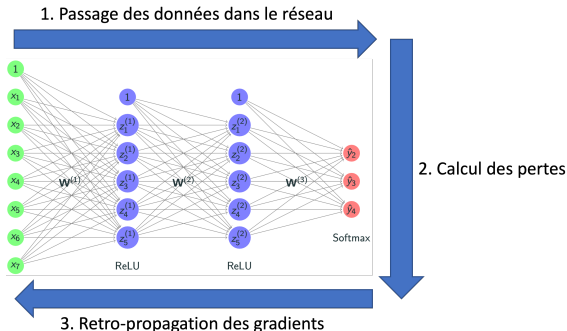
où $w(t)$ est un poids à l'itération t et ν le **taux d'apprentissage**.

Il existe de **nombreux optimiseurs**, qui mettent à jour les poids de manière différente en fonction du gradient et parfois du hessien.

La rétro-propagation

Pour chaque donnée $\mathbf{x}_i$ qui passe dans le réseau, on peut calculer les gradients $\frac{\partial L_i(\mathbf{W})}{\partial \mathbf{w}}$ grâce à la **retro-propagation**. Cette dernière utilise la **règle de dérivation en chaîne** :

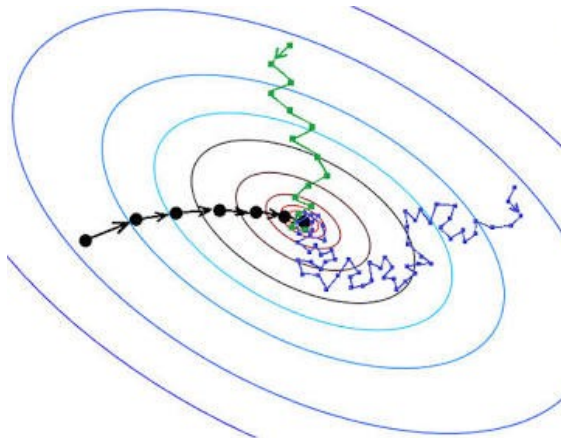
$$\frac{\partial f_2(f_1(x))}{\partial x} = \frac{\partial f_2}{\partial f_1}(f_1(x)) \frac{\partial f_1(x)}{\partial x}$$



Pour des **raisons de performances ET d'optimisation** (ajouter de l'aléatoire aide parfois à éviter les minima locaux) on ne passe pas forcément toutes les données avant de mettre à jour les poids. On a les choix suivants :

- La **descente du gradient en batch**, où l'on fait passer **toutes nos données** avant de mettre à jour les poids.
- La **descente du gradient en mini-batch**, où l'on fait passer un **sous-ensemble** $m < n$ de nos données avant de mettre à jour les poids.
- La **descente du gradient stochastique**, où l'on met à jour les poids après le passage de **chaque individu**.

Mise à jour des poids



Batch GD

- Slowest
- Perfect gradient

Stochastic GD

- Fastest
- Rough-estimate grad

Mini-batch GD

- Compromise

Source : <https://medium.com/@sweta.nit/batch-mini-batch-and-stochastic-gradient-descent-e9bc4cad461>

Dans le terminologie des réseaux de neurones :

- Une **itération** correspond à une mise-à-jour des poids.
- Une **epoch** correspond à un passage de toutes les données.

On aura donc :

- Une itération par epoch pour la descente du gradient en batch.
- $\text{ceiling}(n/m)$ itérations par epoch pour la descente du gradient en mini-batch.
- n itérations par epoch pour la descente du gradient stochastique.